

Concept of Scripting for Upscale and Frame Generator (FSR + IFS) for Weak PCs

1. The Root of the Project (The Need)

Running a heavy game and still recording in high resolution is a task that requires a robust pc and even so these braggers suffer for this double task. So the process between playing and recording is segmented. First, the game is recorded at a quiet resolution so that the gaming pc can use graphics configuration on ultra with the right to ray tracing. After being recorded, the final video is subjected to the upscaling process (sharp increase in resolution) and sometimes to the frame generation process. Some heavy games can be recorded at 30 or 48 fps and then new frames are artificially created. All this using the best of what AI has to offer us in really robust computers.

These solutions largely rely on heavy AI processing, require powerful CPUs and GPUs with dedicated AI cores, and still require high VRAM capabilities that go far beyond what low-end boards can supply.

Do you have a lowend pc? Mine is an AMD FX-6300 and Radeon RX 550 4GB GPU. With a pc like this it is impossible to run the most well-known upscale and frame generation solutions. And of course, we really need these solutions because often the maximum that is possible to record with quality is in 720p at 30fps, a quality that is low by today's standards of video publishing.

2. The Key Turn: I can finally increase the resolution and frames!

The inspiration for the project came from hands-on experience with Lossless Scaling software in the real-time gaming environment. When observing that my weak pc could run the tool to increase the resolution with FSR and generate frames with LSFG to transform a 720p/30fps game into a fluid 1080p/60fps experience without killing the hardware, the following question was raised:

If the principle works lightly in a real-time game, then could it be applied to a previously recorded 720p/30fps video file to upscale to 1080p/60fps or more?

3. A truly lightweight Upscale and Frame Generator is born

To materialize the concept of lightweight video post-processing focused on legacy hardware, the power of consolidated open-source tools was combined with the

development of a mathematical logic running directly on the GPU. Magic is divided into four fundamental pillars:

(ffmpeg) + (Vulkan) + (AMD FidelityFX FSR 1.0) + (IFS - Intersection Fluid Sharpen)

Although there are 4 technologies, they are all merged in the same process that you easily activate via command in your powershell applying to your video the same idea that lossless scaling applies in games running in real time.

- ffmpeg: is the program that generates the videos
- Vulkan: it's dependency, if your vcard supports this technology you're saved.
- AMD FidelityFX: is a shader that runs directly on your GPU doing the upscaling process without using AI.
- IFS (Intersection Fluid Sharpen): it's a shader of my own that was born to generate frames without AI super fast using minimal GPU processing. The idea for the shader comes from vector programs in object intersection processes.

3.1. HARDWARE REQUIREMENTS AND DEPENDENCIES

In order for the post-processing pipeline to work properly on your computer, your system must comply with the following physical and software dependencies:

- **Processor:** Any basic processor with 4 or 6 old cores (such as the AMD FX or Intel Core line from past generations) is perfectly capable of running the pipeline. In the development and testing of this pipeline was used: AMD FX-6300 with 8gb ram (2012 architecture)
- **Video Card:** Your video card must have native support for Vulkan technology. It works on dedicated graphics cards (AMD RX, Nvidia GTX/RTX) and also on integrated chips (AMD Vega or newer Intel HD Graphics). In the development and testing of this pipeline was used: AMD Radeon RX 550 with 4GB vram (Released in 2017)

Even under these severe physical limitations of the test/development computer, the ecosystem was able to deliver processing rates of up to 2.0x (in reduced time) for 1080p/60fps outputs and full stability in 4K renderings.

4. Conclusion and Verdict of Efficiency

The concept proves that intelligent software optimization can bypass severe physical hardware limitations. Replacing heavy AI calculations with

straightforward and cheap mathematical instructions for GPU architecture. That's all lossless scaling offers, but in video post-processing.

*"As the entire upscaling process of this ecosystem is done via **AMD FSR**, the ideal is that the videos are strictly of game gameplay. The algorithm is designed to handle digital graphics, polygon edges, and polishing textures from real rendering. Trying to use videos of real footage, such as movies or cell phone recordings, will break the logic of the process and may not bring the expected result."*

If you also have a lowend pc, come be happy with me!

5. Credits and License Assignments

This ecosystem distributes a modular post-processing pipeline composed of open-source tools integrated with proprietary logic. Below are detailed the copyrights and licenses of each component:

- **Core Component (FFmpeg):** Version *N-125258-gdf94900c98-20260624*. Applicable license: GNU Lesser General Public License (LGPL v2.1 or later) or GNU General Public License (GPL v2.0 or later).
- **Rendering Library (libplacebo):** Built natively into the FFmpeg core binary above. Applicable license: GNU Lesser General Public License (LGPL v2.1 or later).
- **Spatial Scaling Algorithm (FidelityFX FSR):** Version *v1.0.2*. Developed by AMD (*Advanced Micro Devices, Inc.*). Applicable License: MIT (Open-Source, Permissive) License.
- **Frame Generator Algorithm (Intersection Fluid Sharpen - IFS):** Version *v0.1*. Developed by Aless (*MaulSmoke*). Applicable License: MIT (Open-Source, Permissive) License

CREDITS AND TECHNICAL SUPPORT

- **Ideation, Authored** by Shader IFS (MIT) and Pipeline Integration: by Aless (MaulSmoke).

- **Third Party Technologies Used:** FFmpeg (GPL), libplacebo (LGPL), AMD FidelityFX FSR (MIT)

- **Official Project Channel (YouTube):** Watch tutorials, news, performance showcase and ask your questions directly in the channel's community:
<https://youtube.com/@toplayaless?si=OjRmKWfSgl5tDydC>